Blink "Exemplo Trabalhado"

AI-POWERED FINANCIAL ANALYSIS | RAG & LLM WORKFLOWS

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

Blink "Exemplo Trabalhado

Alberto Souza

2026-02-23

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

Contents

O que você vai aprender com este curso? i

Uma análise de investimento confiável exige múltiplas perspectivas simultâneas 1

A estratégia de chunking deve respeitar a natureza do documento 3

Notícias financeiras capturam sinais que relatórios oficiais nunca vão mostrar 5

Parallelization transforma análises de 40 segundos em 20 segundos 7

Prompts especializados transformam um LLM genérico em analistas financeiros distintos 9

Sem filtros de metadados, a busca semântica contamina as análises 11

Índices no Qdrant são pré-requisito para filtros eficientes em collections grandes 13

Output estruturado com Pydantic e Instructor elimina a fragilidade do texto livre 15

Extração inteligente de tickers combina velocidade de mapeamento estático com flexibilidade de LLM . 17

O front-end transforma dados estruturados em decisões visuais e acionáveis 19

Resumo Final 21

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

Sobre este resumo 23

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

O que você vai aprender com este curso?

Imagine ter um analista financeiro pessoal que lê relatórios anuais de centenas de páginas, acompanha notícias em tempo real e cruza tudo isso para gerar uma recomendação de investimento em segundos. Parece ficção? Pois é exatamente isso que você vai construir ao longo deste curso.

O "Exemplo Trabalhado: Mercado Financeiro", ministrado por Daniel Romero, é um projeto prático de ponta a ponta que mostra como combinar **RAG**, **workflows de IA** e **APIs modernas** para criar um sistema real de análise de investimentos. Você vai desde a ingestão de dados brutos da SEC até uma interface visual que exibe recomendações estruturadas de comprar, manter ou vender.

Ao longo destes blinks, você vai descobrir:

- Como construir um **pipeline de análise de investimentos** automatizado com três streams paralelos
- Por que combinar **análise fundamental**, **análise de momento** e **análise de sentimento** produz recomendações mais robustas
- Como ingerir relatórios **10-K** e **10-Q** da SEC em um banco vetorial usando **chunking semântico**
- A diferença entre **chunking semântico** e **chunking linear** e quando usar cada um
- Como buscar e processar **notícias financeiras** do Yahoo Finance com extração limpa de texto
- O poder da **parallelization** como padrão de workflow para executar análises simultâneas com LLMs
- Como usar **filtros de metadados no Qdrant** para garantir que cada busca retorne apenas dados relevantes

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

- Por que **output estruturado** com Pydantic e Instructor transforma texto livre em dados consumíveis
- Como implementar **extração inteligente de tickers** combinando mapeamento estático e LLM como fallback
- Como integrar tudo em um **front-end** Next.js que consome a API e exibe dashboards visuais

Se você quer ver como sistemas de IA são construídos na prática – da ingestão de dados à interface final – este projeto oferece o roteiro completo para transformar teoria em aplicação real no mercado financeiro.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

Uma análise de investimento confiável exige múltiplas perspectivas simultâneas

Se você olha apenas os números financeiros de uma empresa, perde o contexto do mercado. Se olha apenas as notícias, perde os fundamentos sólidos. Se olha apenas o momento atual, perde a trajetória histórica. As plataformas profissionais de análise de investimento combinam várias fontes e múltiplas perspectivas para formar uma tese de investimento – e o sistema que este curso constrói faz exatamente isso, de forma automatizada.

A arquitetura do projeto se baseia em três streams de análise independentes que rodam em paralelo. O primeiro stream, a **análise fundamental**, processa relatórios 10-K da SEC – aqueles relatórios anuais completos que as empresas são obrigadas a publicar. Ele realiza quatro buscas direcionadas: fatores de risco e ameaças competitivas, operações e modelo de negócio, demonstrações financeiras e indicadores de saúde econômica, e a seção MD&A (Management Discussion and Analysis), onde a gestão discute estratégia e perspectivas futuras. O segundo stream foca na **análise de momento**, usando relatórios 10-Q trimestrais para determinar se a empresa está acelerando ou desacelerando. E o terceiro stream realiza **sentiment analysis** sobre notícias recentes, capturando a percepção do mercado em tempo real.

> *"Uma análise de investimento não pode se basear apenas em um único tipo de dado. Se você olha só os números financeiros, perde o contexto do mercado. Se olha só as notícias, perde os fundamentos."*

Depois que os três streams completam, uma etapa de agregação final sintetiza tudo em uma recomendação unificada. Quando

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

duas ou mais streams estão alinhadas, a confiança é alta. Quando os sinais são conflitantes, a confiança é baixa e o sistema explica a divergência. Por exemplo, se os fundamentos são sólidos mas o sentimento é negativo, isso pode indicar oportunidade de compra para longo prazo.

Ao projetar sistemas de análise com IA, pense em perspectivas complementares e não redundantes. Cada stream deve cobrir um aspecto que os outros não cobrem. A combinação de visão de longo prazo (fundamental), médio prazo (momento) e curtíssimo prazo (sentimento) é um framework robusto para qualquer tipo de tomada de decisão.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

A estratégia de chunking deve respeitar a natureza do documento

Nem todos os textos devem ser divididos da mesma forma antes de serem armazenados em um banco vetorial. Relatórios da SEC e notícias financeiras têm estruturas fundamentalmente diferentes, e a estratégia de chunking precisa refletir essa diferença para preservar a qualidade da informação.

Relatórios 10-K e 10-Q da SEC têm uma estrutura complexa, com múltiplos tópicos entrelaçados ao longo de centenas de páginas. Para esses documentos, o curso utiliza **chunking semântico com clustering** – uma abordagem que agrupa parágrafos relacionados tematicamente, mesmo que estejam distantes no texto original. Isso é essencial porque uma mesma informação sobre riscos competitivos pode aparecer em diferentes seções do relatório. Já as notícias financeiras têm uma **estrutura linear natural**: o autor escreveu em uma ordem específica para contar uma história. Quebrar essa linearidade com chunking semântico poderia destruir o fluxo narrativo. Por isso, o curso implementa um **SimpleChunker** – uma classe que divide o texto respeitando a ordem dos parágrafos, apenas garantindo que cada chunk não ultrapasse o limite de 300 tokens.

> *"Notícias têm uma estrutura linear natural, ou seja, o autor escreveu em uma ordem específica para contar uma história. Quebrar essa linearidade com a estratégia de chunking semântico pode perder o fluxo narrativo."*

O SimpleChunker funciona de forma elegante: divide o texto em parágrafos, vai acumulando parágrafos em um chunk enquanto cabem dentro do limite de tokens, e quando adicionar o próximo

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

parágrafo ultrapassaria o limite, finaliza o chunk atual e começa um novo. O tokenizer utilizado é o mesmo modelo de embeddings do projeto (all-MiniLM-L6-v2), garantindo consistência entre a contagem de tokens no chunking e a geração de embeddings.

Antes de definir a estratégia de chunking, analise a estrutura do seu documento. Textos com tópicos entrelaçados pedem chunking semântico. Textos com narrativa linear pedem chunking simples por parágrafos. Usar a estratégia errada pode degradar significativamente a qualidade das buscas no RAG.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

Notícias financeiras capturam sinais que relatórios oficiais nunca vão mostrar

Os relatórios da SEC – 10-K e 10-Q – são documentos estruturados, formais e obrigatórios. Eles trazem dados quantitativos e análises oficiais das empresas. Mas falta um componente essencial para uma análise completa: o **sentimento do mercado**. Notícias financeiras capturam informações que simplesmente não aparecem nos relatórios oficiais – rumores de fusão entre empresas, mudanças na percepção pública, reações a eventos inesperados, análises de especialistas. Tudo isso influencia o mercado e precisa estar disponível no sistema.

Para trazer essas notícias para dentro do pipeline, o curso implementa um **NewsClient** que utiliza a biblioteca `yfinance` (Yahoo Finance) para buscar notícias recentes sobre uma empresa a partir do seu ticker. O cliente filtra apenas stories completas do próprio Yahoo Finance (descartando vídeos, tweets e outros formatos) para manter a consistência no formato de extração. O conteúdo HTML de cada notícia é então processado pelo **Trafilatura**, uma ferramenta especializada em extrair texto limpo de páginas web – ela remove navegação, sidebar, anúncios, rodapé e retorna apenas o texto principal do artigo.

> *"Notícias financeiras capturam informações que não aparecem nos relatórios oficiais, por exemplo, rumores de fusão entre empresas, mudanças na percepção pública, reações a eventos, análises de especialistas. Tudo isso influencia o mercado e precisamos ter isso disponível no nosso sistema."*

Cada notícia extraída é armazenada com metadados ricos: ticker da empresa, título, URL original, data de publicação e source

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

marcado como “Yahoo Finance”. Esses metadados serão fundamentais mais adiante, quando os filtros de busca precisarem separar notícias de relatórios oficiais dentro da mesma collection do Qdrant.

Ao construir pipelines de ingestão de dados, mantenha consistência na estrutura dos metadados entre diferentes fontes. Ter campos padronizados como ticker, source e data permite filtrar e cruzar informações com facilidade, independentemente da origem dos dados.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

Parallelization transforma análises de 40 segundos em 20 segundos

Quando você tem três streams de análise completamente independentes – a fundamental não precisa esperar a de momento, a de sentimento não depende de nenhuma das outras duas – existe um padrão de workflow perfeito para esse cenário: a **parallelization**. Em vez de executar cada análise sequencialmente, as três rodam simultaneamente, e só se juntam no final para a etapa de agregação.

Na implementação prática, o curso utiliza o `asyncio.gather` do Python para executar as três corrotinas em paralelo. Cada stream de análise é uma função assíncrona que faz buscas no Qdrant, monta um prompt especializado e chama o LLM. O `gather` inicia as três análises ao mesmo tempo e aguarda todas completarem, retornando uma tupla com os resultados na ordem em que foram passados. Se cada análise demora cerca de 10 segundos, uma implementação sequencial levaria 40 segundos no total (três análises + agregação). Com parallelization, as três primeiras rodam juntas em 10 segundos, depois a agregação final roda em mais 10 segundos, totalizando 20 segundos – uma redução de 50% no tempo de resposta.

> *"As três streams rodam simultaneamente, todas acontecem ao mesmo tempo. É o workflow perfeito para esse caso, porque as análises são independentes."*

O detalhe técnico crucial é usar o **AsyncGroq** (cliente assíncrono) em vez do cliente síncrono comum. Sem operações verdadeiramente assíncronas, o `gather` executaria as chamadas de forma sequencial disfarçada. O `await` em cada chamada de API garante

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

que a operação seja não-bloqueante, liberando o event loop para processar as outras corrotinas enquanto aguarda a resposta do LLM.

Sempre que tiver tarefas independentes que envolvem chamadas de API ou I/O, considere executá-las em paralelo com `asyncio.gather`. A regra é simples: se uma tarefa não depende do resultado da outra, elas podem rodar ao mesmo tempo. Isso é especialmente impactante em sistemas que fazem múltiplas chamadas a LLMs.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

Prompts especializados transformam um LLM genérico em analistas financeiros distintos

O mesmo modelo de linguagem pode atuar como analista fundamental, analista de momento ou analista de sentimento – tudo depende de como o prompt é construído. No projeto do curso, cada stream de análise recebe um prompt cuidadosamente elaborado que define o papel, os requisitos de análise e o formato esperado da resposta. Essa especialização é o que transforma um LLM genérico em um time de analistas com expertise distinta.

O prompt da **análise fundamental** define o modelo como um "Fundamental Analysis Expert" que trabalha com relatórios 10-K. Ele pede uma tese de investimento de 2 a 3 frases, nota de A a D (com critérios bem definidos), score de confiança, exatamente 3 pontos fortes, 3 preocupações e uma recomendação final de Buy, Hold, Sell ou Avoid. O prompt da **análise de momento** foca no "Momentum Analysis Expert", avaliando mudanças trimestre a trimestre com classificação de Strong, Moderate ou Weak e uma perspectiva de 3 a 6 meses. Já o prompt de **sentiment analysis** atua como "Financial News Analyst", determinando o sentimento geral de 1 a 10 e identificando catalisadores recentes que podem mover a ação.

> *"Para a visualização humana, é até ok, mas tem algumas limitações quando queremos construir aplicações consumindo essa API. O problema é que texto livre não é consumível programaticamente."*

O prompt mais importante é o de **agregação final**, que sintetiza as três análises em uma recomendação unificada. Ele inclui um framework de decisão explícito: quando dois ou mais streams

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

se alinham, a confiança é alta; quando divergem, a confiança é menor e o modelo precisa explicar a divergência. As diretrizes de recomendação também são claras: Buy significa fundamentos fortes mais momento ou sentimento positivo; Hold indica sinais mistos; Sell aponta riscos significativos.

Ao criar prompts para análise especializada, seja extremamente específico nos requisitos. Definir escalas numéricas precisas (0 a 1, A a D), listas com quantidade exata de itens (3 pontos fortes, 3 preocupações) e frameworks de decisão explícitos reduz a ambiguidade e torna as respostas mais consistentes entre chamadas.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

Sem filtros de metadados, a busca semântica contamina as análises

Um dos problemas mais sutis e perigosos em sistemas de RAG é a **contaminação cruzada de dados**. Quando você faz uma busca semântica por "AAPL Financial Performance", o sistema pode retornar chunks sobre a Apple, mas também chunks sobre a NVIDIA – se eles forem semanticamente similares. Um chunk falando sobre "performance financeira excepcional de semicondutores" pode ter alta similaridade vetorial com a query, mesmo sendo sobre uma empresa completamente diferente. O embedding não sabe que você quer apenas informação da Apple; ele só sabe que você está buscando sobre performance financeira.

O problema vai além da mistura de empresas. Sem filtros, uma query de momento que deveria buscar apenas em relatórios 10-Q pode acabar retornando chunks de 10-K ou até de notícias. E a query de sentimento, que deveria processar apenas notícias, pode trazer chunks de relatórios oficiais da SEC. Cada stream foi projetado para analisar um tipo específico de dado, e essa mistura degrada a qualidade das recomendações. A solução é adicionar **filtros de metadados** (Payload Filtering) nas buscas do Qdrant. O Qdrant aplica esses filtros **antes** de calcular similaridades vetoriais, o que significa que ele nunca vai considerar pontos que não atendem às condições – melhorando tanto a qualidade dos resultados quanto a performance.

> *"Atualmente, quando fazemos as buscas no Qdrant, estamos usando apenas buscas semânticas, sem nenhum filtro por metadados. Isso pode causar um problema na qualidade das análises."*

A estratégia implementada no curso é precisa: o stream funda-

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

mental filtra por `ticker` e `form_type = 10-K`, garantindo que só busca em relatórios anuais da empresa analisada. O stream de momento filtra por `ticker` e `form_type = 10-Q`, pegando apenas relatórios trimestrais. E o stream de sentimento filtra por `ticker` e `source = Yahoo Finance`, isolando apenas notícias. Cada análise trabalha exclusivamente com os dados corretos.

Em qualquer sistema de RAG com múltiplas fontes de dados na mesma collection, filtros de metadados não são opcionais – são obrigatórios. Sem eles, a busca semântica vai eventualmente retornar documentos irrelevantes que "parecem" relevantes. Planeje seus metadados desde a ingestão, e crie índices nos campos que pretende filtrar.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

Índices no Qdrant são pré-requisito para filtros eficientes em collections grandes

Criar filtros de metadados sem índices é como procurar um nome em uma lista telefônica que não está em ordem alfabética. Sem índices, o Qdrant precisa escanear **todos os pontos** da collection para aplicar o filtro, tornando a operação lenta e inviável em collections grandes. Com índices, a filtragem fica muito mais rápida porque o Qdrant consegue localizar diretamente os pontos relevantes.

No projeto, a criação de índices é feita por um script separado (`createIndexes.py`) que só precisa ser executado uma vez – os índices ficam salvos permanentemente na collection. São criados três índices do tipo `keyword` para os campos `metadata.ticker`, `metadata.form_type` e `metadata.source`. O tipo keyword é ideal para valores exatos como strings curtas (tickers, tipos de formulário, nomes de fontes). O script utiliza um loop simples que percorre uma lista de campos e chama `create_payload_index` para cada um, tornando fácil adicionar novos índices no futuro.

> *"Para usar filtros de forma eficiente, especialmente em collections grandes, você precisa criar índices nos campos do payload que vai filtrar. Sem índices, o Qdrant precisa escanear todos os pontos da collection para aplicar o filtro, o que torna a operação bem lenta."*

Os filtros no Qdrant usam uma estrutura com condições lógicas. A condição `must` funciona como um AND lógico – todas as condições dentro dela precisam ser verdadeiras simultaneamente. Além do `must`, existem `should` (funciona como OR, onde pelo menos uma condição precisa ser verdadeira) e `must_not` (funciona como negação, excluindo pontos que atendem à condição).

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

No projeto, apenas o `must` é utilizado, porque a necessidade é garantir que tanto o ticker quanto o tipo de documento sejam exatamente os esperados.

Trate a criação de índices como parte do pipeline de ingestão, não como um passo opcional. Crie um script dedicado que indexa todos os campos usados em filtros e execute-o uma vez após a ingestão inicial. Isso evita surpresas com erros ou lentidão quando os filtros forem ativados na aplicação.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

Output estruturado com Pydantic e Instructor elimina a fragilidade do texto livre

Quando um LLM retorna análises como texto livre, cada resposta pode vir formatada de um jeito diferente. Você não consegue garantir que o score de confiança esteja sempre entre 0 e 1, que a análise tenha exatamente 3 pontos fortes, ou que a recomendação seja sempre uma das opções válidas (Buy, Hold, Sell, Avoid). Isso impossibilita a construção de dashboards, alertas automáticos e comparações estruturadas entre empresas. A solução é usar **output estruturado** com modelos Pydantic validados.

O curso implementa modelos Pydantic detalhados para cada tipo de análise. O `FundamentalAnalysis` define campos como `investment_gr` (Literal com opções A, B, C, D), `confidence_score` (float entre 0 e 1 usando `Field(ge=0, le=1)`), `key_strengths` (lista com exatamente 3 strings usando `min_length=3, max_length=3`) e `recommendation` (Literal com Buy, Hold, Sell, Avoid). A mesma precisão é aplicada ao `MomentumAnalysis` e ao `SentimentAnalysis`. Para conectar esses modelos ao LLM, o curso utiliza a biblioteca **Instructor**, que integra Pydantic com o cliente Groq. Ao envolver o cliente com `instructor.from_groq()` e passar o `response_model` na chamada de completion, o Instructor garante que a resposta respeita a estrutura definida – e se o LLM retornar algo inválido, tenta novamente automaticamente.

> *"Texto livre não é consumível programaticamente. Se você quiser extrair apenas o score de confiança da análise fundamental ou pegar só a recomendação final, você precisa fazer parsing de texto que pode vir em formatos diferentes a cada requisição."*

O resultado é uma API que retorna objetos JSON com campos es-

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

pecíficos, tipos validados e restrições definidas. O `AgentResponse` agora contém objetos Pydantic completos em vez de strings, permitindo que qualquer front-end consuma os dados de forma confiável. O Pydantic valida automaticamente tipos, intervalos numéricos e formatos, eliminando a necessidade de parsing manual ou instruções de JSON nos prompts.

Sempre que uma resposta de LLM será consumida por outra aplicação (front-end, API, pipeline de dados), use output estruturado. Bibliotecas como Instructor tornam isso trivial: defina o modelo Pydantic com as restrições desejadas e passe como `response_model`. Isso é especialmente importante em produção, onde consistência e validação são indispensáveis.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

Extração inteligente de tickers combina velocidade de mapeamento estático com flexibilidade de LLM

Exigir que o usuário saiba o ticker exato de uma empresa para fazer uma consulta – como AAPL para Apple ou MSFT para Microsoft – cria uma barreira de usabilidade significativa. A maioria das pessoas pensa em nomes de empresas, não em símbolos de ações. O sistema precisa ser capaz de interpretar "como está a Apple hoje?" e automaticamente identificar que o ticker correto é AAPL.

A solução implementada no curso usa uma estratégia de **duas camadas**. A primeira camada é um mapeamento estático – um dicionário Python simples que associa nomes de empresas comuns (em lowercase) aos seus tickers. Quando o usuário envia uma query, o sistema converte o texto para minúsculas e verifica se algum nome de empresa do dicionário aparece na frase. Essa busca é instantânea e não custa nada. A segunda camada é um **LLM como fallback**: quando o mapeamento estático não encontra correspondência, o sistema envia a query para o modelo com um prompt especializado em extração de tickers. O prompt inclui regras claras (retornar apenas o símbolo, retornar NONE se não houver empresa mencionada) e exemplos few-shot para guiar o modelo.

> *"O usuário não precisa saber os símbolos das empresas. Agora pode simplesmente fazer perguntas em linguagem natural sobre as empresas, e o mapeamento estático entra em ação para os casos mais comuns, e o LLM como fallback cobre os casos mais inusitados."*

A validação do ticker extraído é garantida por um modelo Pydan-

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

tic com uma expressão regular (`^[A-Z]{1,5}$`) que aceita apenas de 1 a 5 caracteres maiúsculos – o formato padrão dos tickers americanos. Se o LLM retornar algo inválido, o Instructor automaticamente tenta novamente até obter uma resposta que passe na validação. Quando nenhuma empresa é identificada (por exemplo, em uma pergunta como "como está o tempo hoje?"), o sistema levanta um erro HTTP 400 Bad Request com uma mensagem explicativa, funcionando como uma camada de proteção contra alucinações.

A estratégia de mapeamento estático + LLM como fallback é um padrão reutilizável para qualquer cenário onde você precisa de extração de entidades. O mapeamento cobre os casos comuns com custo zero e latência mínima. O LLM cobre os casos raros com flexibilidade máxima. Combine os dois para ter o melhor de ambos os mundos.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

O front-end transforma dados estruturados em decisões visuais e acionáveis

Um back-end poderoso sem interface visual é como um analista brilhante que nunca apresenta seus relatórios. O último passo do projeto conecta toda a infraestrutura construída – FastAPI, RAG, workflow paralelo, output estruturado – a uma interface front-end em **Next.js** com TypeScript e Tailwind CSS que torna as análises acessíveis e consumíveis visualmente.

A arquitetura de integração usa **rewrites do Next.js** como proxy reverso: quando o front-end faz uma requisição para `/api/agent`, o Next.js redireciona internamente para `localhost:8000/agent` no FastAPI. Isso simplifica a configuração de CORS e centraliza as chamadas de API. No lado do back-end, o FastAPI precisa do middleware CORS configurado para aceitar requisições da origem do front-end. A interface oferece dois modos: o modo **RAG simples**, que faz uma busca direta nos documentos e retorna uma resposta em Markdown, e o modo **Multistream Analysis** (Agent), que aciona os três streams paralelos e exibe a análise completa com cards visuais para cada perspectiva – fundamental, momento e sentimento – além de painéis de riscos e oportunidades lado a lado.

> *"Agora temos uma aplicação completa, de ponta a ponta. Back-end FastAPI com RAG, busca híbrida, sistema de agent com análise paralela de três streams e output estruturado validado. Além da extração razoavelmente inteligente de tickers e um front em JavaScript consumindo tudo de forma visual."*

Um detalhe importante de design é que o modo Agent é ativado explicitamente por um checkbox, e não de forma automática. Essa

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

é a mesma abordagem adotada pelos maiores providers – Claude, ChatGPT – onde modos especiais como "thinking" ou "deeper research" são ativados manualmente. O motivo é pragmático: quando o modelo tenta decidir sozinho quando usar o modo Agent, ele pode alucinar, inventando respostas como se tivesse feito a análise completa quando na verdade não fez.

Quando construir interfaces para sistemas de IA com múltiplos modos de operação (RAG simples vs. Agent com análise profunda), dê ao usuário controle explícito sobre qual modo usar. A ativação manual evita alucinações silenciosas e permite que o usuário entenda o nível de análise que está recebendo.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

Resumo Final

Este curso demonstra, do início ao fim, como construir um sistema de análise de investimentos automatizado combinando RAG, workflows de IA e APIs modernas. Aqui está o que você aprendeu:

Mensagem Central: Sistemas de IA em produção não são apenas modelos de linguagem respondendo perguntas – são arquiteturas completas que combinam ingestão inteligente de dados, busca vetorial com filtros precisos, múltiplos streams de análise em paralelo e output estruturado validado. O valor real está na orquestração de todas essas peças trabalhando juntas.

A jornada começa com a compreensão de que uma análise de investimento confiável exige **múltiplas perspectivas** – fundamental, momento e sentimento – cada uma alimentada por fontes de dados distintas. A **estratégia de chunking** deve respeitar a natureza do documento: semântico para relatórios complexos da SEC, linear para notícias com narrativa fluida. As notícias do **Yahoo Finance** trazem sinais de mercado que relatórios oficiais nunca capturam. A **parallelization** com `asyncio.gather` corta o tempo de resposta pela metade ao executar streams independentes simultaneamente.

Na parte de qualidade e precisão, **filtros de metadados no Qdrant** eliminam a contaminação cruzada entre empresas e tipos de documento, e **índices** são pré-requisito para que esses filtros funcionem de forma eficiente. O **output estruturado** com Pydantic e Instructor transforma texto livre em dados consumíveis por qualquer aplicação. A **extração inteligente de tickers** com mapeamento estático e LLM como fallback torna o sistema acessível para usuários que pensam em nomes de empresas, não em sím-

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

bolos de ações. E o **front-end** em Next.js fecha o ciclo, transformando dados estruturados em dashboards visuais e acionáveis.

Conselho prático

Ao construir sistemas de IA para produção, invista mais tempo na arquitetura de dados e na qualidade da ingestão do que na engenharia de prompts. Os prompts são importantes, mas a qualidade da resposta de um LLM depende fundamentalmente da qualidade dos dados que ele recebe como contexto. Filtros de metadados, estratégias de chunking adequadas e validação de output são o que separam um protótipo de um sistema confiável.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00

Sobre este resumo

Este resumo foi gerado a partir do curso **"Exemplo Trabalhado: Mercado Financeiro"** de **Daniel Romero**, parte da formação em Inteligência Artificial e Retrieval Systems.

O módulo apresenta um projeto prático completo de análise de investimentos automatizada, cobrindo ingestão de relatórios 10-K e 10-Q da SEC e notícias do Yahoo Finance, busca vetorial híbrida no Qdrant com filtros de metadados, workflow de parallelization com três streams de análise (fundamental, momento e sentimento), output estruturado com Pydantic e Instructor, extração inteligente de tickers e integração com front-end Next.js – tudo construído sobre uma API FastAPI.

Este blink foi criado para oferecer uma visão rápida e prática dos principais conceitos do curso. Para aprofundamento com implementação completa e código funcional, recomendamos o curso completo.

Luiz Paulo Rodrigues Almeida - luiz.physics@gmail.com - CPF: 347.774.058-00